# Compliance Aware, Agentic Pitch Book Generation For Bankers

A template-aware PowerPoint generation system with agentic data retrieval for investment banking.

A Queen Mary Final Year Dissertation

**Andrei Rizea**

Student ID: 220300153

Supervised by: Manesha Peiris

Programme of Study:

Digital & Technology Solutions (Software Engineering)

Module: IOT635W

Queen Mary University of London

February 16, 2026

# Contents

# List of Figures

# List of Tables

# Introduction, Scope and Context

Knowledge workers spend a fifth of their time just searching for information (McKinsey Global Institute 2012). Investment banking (IB) is more affected than most. Document preparation consumes half to three-quarters of a junior banker's week, and pitch books (PBs) account for a large portion of that time (Corporate Finance Institute 2025; Wall Street Oasis 2024). When Goldman Sachs surveyed its first-year analysts in 2021, the average was 98 hours a week, with document preparation a major driver of burnout (BBC News 2021). Yet most PBs look nearly identical. Change the company name and update the numbers, and you could reuse last month's deck. This repetitiveness makes them good candidates for automation.

IB firms advise companies on mergers, acquisitions, capital raising, and other financial transactions. PBs are the primary vehicle for communicating analysis to clients and internal stakeholders. They summarise a company's financial position, compare it to peers, and make the case for a particular transaction. Getting these documents right matters, but getting them done quickly matters too.

Firms have tried internal document libraries, dedicated formatting specialists, and automation tools. None have fully addressed the underlying issue: too much skilled human time goes toward work that does not require skill.

### Where Analyst Time Goes

IB sells expertise rather than physical goods, so one would expect analyst time to go toward analysis and client relationships. Often it does not.

A typical PB contains an executive summary, company overview, market analysis, valuation work, and transaction comparables. These sections appear in nearly every deck. The data inside changes, but the shell around it rarely does.

Analyst tasks fall into two buckets. The first holds work that genuinely needs a trained banker: modelling, valuation, and client advisory. The second holds everything else: hunting down data, reformatting slides, and fixing fonts. Too many hours go into the second bucket.

The process follows a predictable pattern: dig through document archives for similar deals, pull financial data from Bloomberg and public filings, copy it into PowerPoint, reformat everything to match the firm's visual standards, then send it up for review. Senior bankers mark it up, and the cycle repeats until the document is ready. Every step has friction. Archive searches are slow, data entry is error-prone, and formatting is tedious.

Banks pay junior analysts well, so time on low-skill tasks is a poor return on that investment. Radhakrishna et al. (2024) found that well-designed knowledge management systems boosted productivity by 20 to 25 percent, with document production identified as a prime candidate for automation.

The numbers back this up. McKinsey Global Institute (2012) found that the average knowledge worker spends 28% of their workweek managing email and nearly 20% searching for internal information. APQC research found that a quarter of knowledge workers' time is lost to productivity drains including duplicated work and information retrieval (APQC 2023).

Figure 1 illustrates these productivity drains. IB analysts likely fare worse than these averages given the document-intensive nature of the work.

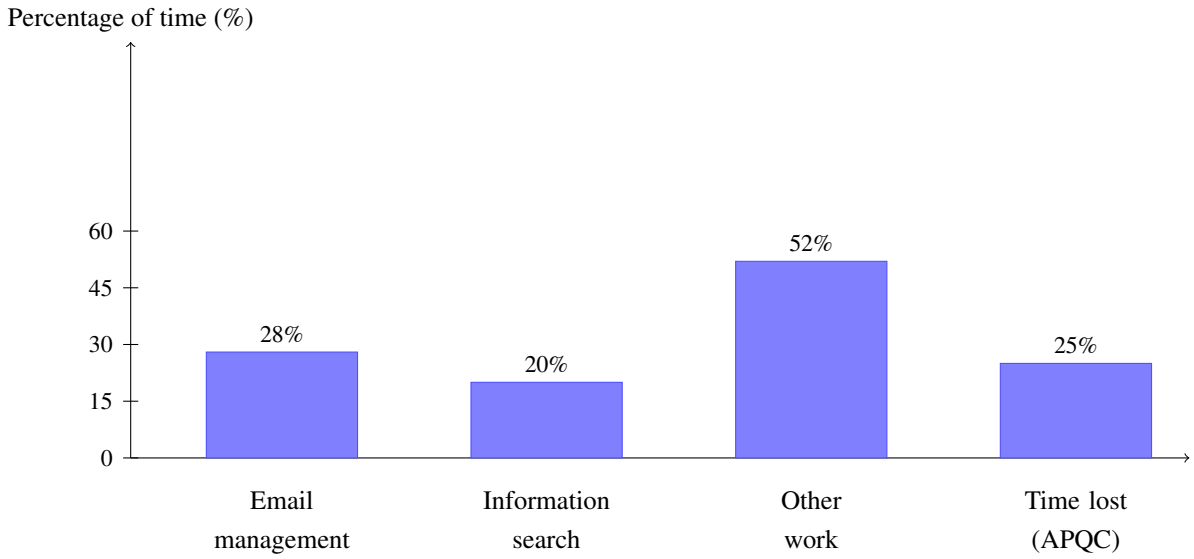


Figure 1: Knowledge worker time allocation based on McKinsey Global Institute (2012) and APQC (2023) research. The "other work" category represents the remaining time after email and search activities.

In short, banks hire analysts for their analytical skills but spend a large share of their time on formatting and data entry. Existing tools have not solved this because they either miss IB visual standards or require too much manual work. Every hour on low-value tasks is an hour not spent on the analysis that differentiates one bank from another.

## Problem Analysis

Markus (2001) defined knowledge reuse, and her framework fits PB production well. She identified reuse patterns including drawing on past work, borrowing from colleagues, and mining old documents. PB creation involves all of these. However, three obstacles prevent effective reuse in practice.

The first obstacle is that too much time goes to low-value work. Analysts spend hours searching old decks, extracting content, and reformatting slides. None of that requires analytical thinking, yet it crowds out the work that does.

The second obstacle is that institutional knowledge tends to disappear. Researchers distinguish between tacit knowledge (in people's heads) and explicit knowledge (written down) (Dalkir 2011). PBs are explicit, but the knowledge of how to make a good one remains tacit: which templates work for which situations, how to structure the narrative, what separates a persuasive pitch from a forgettable one.

The third obstacle is that onboarding takes longer than it should. Each bank has its own templates, data sources, and quality expectations. Junior analysts absorb these through trial and error, which is slow for them and a drain on the senior bankers who review their work.

These problems reinforce each other. Scattered knowledge wastes search time, tacit standards cause inconsistent output, and informal training drains senior staff. A system that addressed even one of these would help.

## Gap Analysis

Automated presentation tools have existed for years. Early versions converted text to slides using layout algorithms, but visual quality was poor (Zheng et al. 2025). Template-based systems fixed the visual problems but made everything rigid. PPTAgent (Zheng et al. 2025) takes a different approach: it analyses reference presentations, extracts structural patterns, and applies them to new content. It beat older text-to-slide systems on both content and design quality in testing.

The limitation is that PPTAgent targets general-purpose presentations. IB PBs have financial tables, transaction comparables, and market charts that follow specific visual conventions. Getting the template wrong undermines credibility. Commercial tools like Beautiful.ai, Tome, and Gamma share this flaw: the output is generic, every slide needs rework, and none connect to financial data sources. IB PBs require both domain knowledge and visual precision, and no existing tool handles both.

Current systems either produce generic output that needs heavy cleanup, or they require large training datasets most banks lack. No existing tool connects slide generation to financial APIs. Data retrieval and document generation remain separate manual steps.

Agentic AI architectures look promising (IBM 2024). Instead of one model doing everything in a single pass, the system breaks tasks into steps and uses the right tool for each one. Wang et al. (2024) surveyed LLM-based agents and found rapid progress. McKinsey & Company (2024) estimates generative AI could deliver $340 billion annually in banking.

This project addresses that gap. The pipeline extracts layouts, colour palettes, and placeholder positions from a template using python-pptx, uses an LLM to plan content, then assembles slides matching the original style. The system uses existing agentic frameworks rather than custom orchestration. RAG against past PBs is a stretch goal.

The approach differs from prior work in two ways. First, it starts with visual constraints and works backward to content, rather than generating content and hoping the formatting works. Second, it uses an agentic architecture where specialised components handle distinct parts of the task, making the system easier to test and modify.

## Aims and Objectives

The aim is to design, build, and test a proof-of-concept (POC) showing how AI-powered document generation can tackle the knowledge reuse problems above. The system must respect institutional templates throughout. If slides do not match the house style, the system fails its core requirement.

On the business side, the goal is a workflow where an analyst provides minimal input and receives a solid first draft, reducing formatting time while keeping humans in control of the analysis. Target users are junior analysts.

On the learning side, the project explores how to connect LLM orchestration with programmatic document generation: agentic architectures, template pattern extraction, and end-to-end AI pipeline design. Human-in-the-loop (HITL) design is central, since AI tools in regulated industries need to support professional judgement rather than replace it.

Five objectives structure the work:

1. Build a template analysis component that takes a reference PowerPoint file and extracts its layout structures, colour palettes, font specifications, and placeholder positions. Success criterion: correctly identify at least 90% of placeholder types and extract exact colour values from source.
2. Build an agentic data retrieval layer that automatically pulls company financials, market data, and regulatory filings from Yahoo Finance and SEC EDGAR. Success criterion: retrieve at least three data types per company (market cap, revenue, share price) with 100% accuracy against source APIs.
3. Build an LLM-powered content planning module that produces structured slide specifications adapted to different PB types while keeping the overall narrative coherent. Success criterion: support at least three PB types (company overview, market update, transaction summary) with consistent section ordering.
4. Build a slide assembly component that takes the content plan and template patterns and produces a PowerPoint file meeting formatting standards. Success criterion: output files match template colours exactly, fonts exactly, and placeholder positions within 5% tolerance.
5. Evaluate the system by measuring generation speed, template matching, and content quality. Success criterion: generate a complete 10-slide PB draft in under 5 minutes with template fidelity score exceeding 85%.

## Scope and Success Criteria

The POC takes user input (company, transaction type, dates, reference template, PB type) and produces a formatted PowerPoint. Target PB types are company overviews, market updates, and transaction summaries. Full valuation presentations are out of scope.

Design choices: agentic frameworks rather than custom architecture, programmatic template extraction with python-pptx rather than vision models, public APIs only, RAG as a stretch goal. Out of scope: real-time data feeds, production infrastructure, compliance validation. Practical limits: one academic term, standard hardware, student API budget.

Success criteria:

- Generate complete PB draft within five minutes
- Match reference template formatting
- Accurate financial data from source APIs
- Content structure following IB conventions, verified against precedent examples

One principle runs through all of this: HITL design. The system produces drafts for human review, not finished products. AI can handle the mechanical work; the analytical judgement stays with the banker.

# Methodology and Project Plan

This section covers the development approach, requirements, technology choices and their rationale, the project schedule, and considerations around risks and ethics. Reproducibility matters throughout. Anyone reading this work should be able to understand the choices made, replicate the setup, and reproduce the results.

## Development Methodology

Software development methodologies range from plan-driven approaches like Waterfall to adaptive approaches like Agile (Sommerville 2016). Choosing the right methodology depends on requirement stability, project complexity, stakeholder availability, and team size. Table 1 compares methodologies against criteria relevant to this project.

| **Criteria** | **Waterfall** | **Scrum** | **Iterative Prototyping** |
|---|---|---|---|
| Requirements stability | Requires fixed requirements upfront | Accommodates changing requirements | Works well when requirements change |
| Feedback loops | Late feedback after implementation | Sprint reviews every 2-4 weeks | Continuous feedback through prototypes |
| Risk management | Risks discovered late | Regular risk reassessment | Early risk identification through prototypes |
| Solo developer suitability | Moderate | Low (designed for teams) | High |
| Research integration | Poor | Moderate | Excellent |

Table 1: Comparison of Development Methodologies

The project uses a hybrid methodology combining iterative prototyping with structured research engineering practices. This approach draws on Boehm (1988)'s spiral model, which emphasises risk-driven iteration and early prototyping. Pure Waterfall cannot accommodate the exploratory nature of this work. Full Scrum adds overhead without benefit for a solo developer. Iterative prototyping allows adaptation as understanding develops, while structured practices keep the work repeatable.

Each two-week iteration will produce a working prototype targeting defined requirements. At each cycle's end, a retrospective review will document outcomes. Feedback from IB practitioners, where possible, will shape subsequent cycles.

The project uses Git with a simplified GitFlow model (main, develop, feature branches). Continuous integration via GitHub Actions will run tests and linting on every push, enforcing 80% coverage for core

modules (Fowler and Foemmel 2006). Architecture Decision Records (ADRs) will document technical choices and trade-offs (Tyree and Akerman 2005), creating an audit trail that explains not just what was built but why.

## Requirements Analysis

Following established requirements engineering practice (Robertson and Robertson 2012), this section separates business requirements (the value delivered) from functional requirements (what the system does) and non-functional requirements (how well it performs). Requirements were gathered from the problem analysis, financial services experience, and published research on document generation and agentic AI.

Requirements follow MoSCoW prioritisation (Clegg and Barker 1994). "Must have" requirements define the minimum viable product. "Should have" and "Could have" add functionality if time permits. "Won't have" requirements clarify scope boundaries.

### Business Requirements

Business requirements describe what value the project should deliver:

1. **BR1: Productivity Enhancement**: Reduce time spent on initial PB drafting by generating solid first drafts from minimal input.
2. **BR2: Knowledge Codification**: Capture institutional presentation standards through template analysis, reducing reliance on tacit knowledge.
3. **BR3: Quality Consistency**: Follow corporate visual identity standards consistently, reducing revision cycles caused by formatting issues.
4. **BR4: Data Accuracy**: Financial data in generated PBs should accurately reflect source information.

### Functional Requirements

Table 2 lists functional requirements by component using MoSCoW prioritisation (Must have, Should have, Could have, Won't have).

| ID | Component | Requirement | Priority |
|---|---|---|---|
| FR1 | Template Analyser | Extract slide layouts from reference .pptx files | Must |
| FR2 | Template Analyser | Identify colour palettes and font specifications | Must |
| FR3 | Template Analyser | Map placeholder positions and content types | Must |
| FR4 | Data Retrieval | Fetch company financials from Yahoo Finance API | Must |
| FR5 | Data Retrieval | Retrieve SEC filings via EDGAR API | Should |
| FR6 | Data Retrieval | Perform web searches for company news | Should |
| FR7 | Content Planner | Generate slide-by-slide content specifications | Must |
| FR8 | Content Planner | Adapt content structure to PB type | Must |
| FR9 | Content Planner | Maintain narrative coherence across slides | Should |
| FR10 | Slide Builder | Assemble slides matching template layouts | Must |
| FR11 | Slide Builder | Populate placeholders with generated content | Must |
| FR12 | Slide Builder | Generate charts from financial data | Could |
| FR13 | Orchestration | Coordinate multi-step generation workflow | Must |
| FR14 | Orchestration | Handle API failures gracefully | Should |
| FR15 | RAG Search | Query precedent material repository | Could |

Table 2: Functional Requirements Specification

### Non-Functional Requirements

Table 3 lists quality requirements with measurable acceptance criteria.

| ID | Category | Requirement | Acceptance Criterion |
|---|---|---|---|
| NFR1 | Performance | System generates complete PB draft | Within 5 minutes of input submission |
| NFR2 | Performance | API response handling | Timeout after 30 seconds with graceful degradation |
| NFR3 | Reliability | System availability during demonstration | 95% uptime during evaluation period |
| NFR4 | Usability | Input specification interface | Requires no technical expertise to operate |
| NFR5 | Maintainability | Codebase documentation | All public functions documented with docstrings |
| NFR6 | Security | API credential management | No credentials in source code, environment variables used |
| NFR7 | Security | Input validation | All user inputs sanitised before processing |
| NFR8 | Compatibility | Output format | Valid .pptx files openable in Microsoft PowerPoint |

Table 3: Non-Functional Requirements Specification

## Technology Stack

Technologies were chosen based on four factors:

- **Task fit**: Does the tool solve the specific problem well?
- **Familiarity**: Is there existing experience with the tool?
- **Maturity**: Is it well-documented with active community support?
- **Maintainability**: Will it be easy to debug and extend?

### Programming Language and Framework

Python was chosen as the main language because it dominates AI and ML development and has a large library ecosystem. Familiarity with Python from work experience means the focus can remain on the problem rather than learning a new language. Table 4 compares Python with alternatives.

| **Criteria** | **Python** | **JavaScript/Node.js** | **Java** |
|---|---|---|---|
| Benefits | Best AI/ML libraries, mature PowerPoint library (python-pptx), rapid development | Native async, same language frontend and backend | Strong typing, enterprise support |
| Risks | Slower execution than compiled languages, GIL limits true parallelism | AI/ML ecosystem less mature, limited PowerPoint options | Verbose code, slower development |
| AI/ML support | Excellent (PyTorch, LangChain, OpenAI SDK) | Growing but limited | Moderate (DJL, Langchain4j) |
| PowerPoint manipulation | python-pptx (mature, well-documented) | Limited options | Apache POI (verbose API) |

Table 4: Programming Language Comparison

FastAPI will handle the backend. Its async request handling lets the system call multiple external data sources in parallel. It also generates OpenAPI documentation automatically, providing a clear contract for how the API behaves.

### AI and LLM Integration

The system will use LLMs for content planning and generation. Table 5 compares the options considered.

| Criteria | GPT-4/GPT-4o | Claude 3.5 | Gemini Pro |
|---|---|---|---|
| Benefits | Best structured output, mature agent SDK, extensive documentation | Largest context window (200K), strong reasoning | Massive context (1M tokens), competitive pricing |
| Risks | Higher cost per token, rate limits on free tier | Newer agent tooling, less community examples | Less mature structured output, fewer agent examples |
| Structured output | Excellent (native JSON mode) | Good | Good |
| Function calling | Native, well-documented | Native support | Native support |
| Agent frameworks | OpenAI Agents SDK (mature) | Claude Agent SDK | Primarily via LangChain |

Table 5: Large Language Model Comparison

OpenAI's GPT-4o was chosen as the primary LLM. It handles structured output well, supports tool-use patterns through the OpenAI Agents SDK, and balances capability against cost for a student budget. GPT-4o will be the baseline for evaluation.

### Data Sources and APIs

The project uses publicly accessible APIs to avoid proprietary data licensing complications. Proprietary data sources like Bloomberg or Refinitiv would provide richer data but come with licensing restrictions that complicate academic work. Public APIs provide enough data to demonstrate the concept while keeping the project reproducible by others. Table 6 summarises the data sources.

| Source | Benefits | Risks | Access |
|---|---|---|---|
| Yahoo Finance | Free, comprehensive financial data, easy Python library (yfinance) | Unofficial API may change without notice, data delays | yfinance library |
| SEC EDGAR | Official regulatory filings, reliable, free | US companies only, complex document parsing required | REST API (10 req/s) |
| Web Search | Current news, market commentary | Cost per query, relevance filtering needed | SerpAPI or similar |

Table 6: External Data Sources

### Frontend and Deployment

Table 7 compares frontend framework options.

| **Criteria** | **Next.js** | **React + TypeScript** | **Vue.js** | **Svelte** |
|---|---|---|---|---|
| Benefits | Built-in API routes, server-side rendering, industry standard, strong community | Large ecosystem, strong typing, excellent tooling | Gentle learning curve, good documentation | Smaller bundle size, less boilerplate |
| Risks | Opinionated structure, Vercel-centric tooling | Boilerplate overhead, frequent ecosystem changes | Smaller job market, fewer enterprise examples | Immature ecosystem, limited component libraries |
| Component libraries | All React libraries (MUI, Chakra, Radix) | Extensive (MUI, Chakra, Radix) | Good (Vuetify, Quasar) | Limited options |
| TypeScript support | Native, excellent | Native, excellent | Good | Good |
| Deployment options | Vercel (optimised), Netlify, AWS | Vercel, Netlify, AWS | Similar options | Similar options |

Table 7: Frontend Framework Comparison

Next.js was chosen for its built-in API routes, native TypeScript support, and access to the full React component library ecosystem.

Table 8 compares deployment and hosting options.

| **Criteria** | **Vercel + Render** | **AWS** | **Self-hosted** |
|---|---|---|---|
| Benefits | Zero configuration, automatic deployments, free tiers suitable for POC | Full control, scalable | Complete control, no vendor lock-in |
| Risks | Vendor lock-in, cold starts on free tier | Complex setup, cost management overhead | Time-consuming setup, maintenance burden |
| Cost for POC | Free tier sufficient | May incur charges | Server costs |
| Setup time | Minutes | Hours to days | Days |

Table 8: Deployment Platform Comparison

The frontend will deploy through Vercel, the Python backend on Render, and Supabase will provide PostgreSQL. These choices prioritise development speed over infrastructure control.

### Reproducibility

Reproducibility means someone else can take this work, run it, and get the same results (Runeson et al. 2012). LLM-based systems make this harder than traditional software since the same prompt can produce different outputs across runs. Identical results cannot be guaranteed, but it is possible to get close.

To support this, all Python dependencies will be pinned to exact versions, and a Dockerfile will be provided so others can replicate the environment. For evaluation runs, LLM temperature will be set to zero to reduce randomness. Configuration will be separated from code using a config file, with all settings logged at startup. Each evaluation run will be tied to a specific Git commit, and the full prompts and responses for LLM calls will be logged so that output generation can be traced.

### Evaluation Methodology

The evaluation follows the Goal-Question-Metric (GQM) paradigm (Basili et al. 1994), which structures measurement around explicit goals. The primary goal is to assess whether the system reduces document preparation time while maintaining quality standards. Following guidelines for experimentation in software engineering (Wohlin et al. 2012), the evaluation uses both quantitative metrics and qualitative assessment.

The system will be assessed against four metrics:

- **Efficiency**: target under 300 seconds for a 10-slide deck
- **Template fidelity**: programmatic comparison of layout, colour, and font properties
- **Data accuracy**: verification against source APIs
- **Content quality**: rubric-based assessment of narrative coherence and IB conventions

To contextualise performance, outputs will be compared against a naive baseline using generic tools (Gamma, Beautiful.ai) and estimated manual creation time. This comparison shows whether the system actually improves on existing approaches.

Ablation studies will test each component's contribution. The system will be run with different components disabled: content generation without template analysis, template analysis without LLM planning, and data retrieval without web search. These experiments will reveal which parts of the pipeline contribute most to output quality.

Evaluation will use fixed test cases covering different company types, sectors, and PB types. The test set will include large-cap technology companies, mid-cap manufacturing firms, and small-cap financial

services companies. Each test case will be documented with input parameters and expected outputs to support replication.

For qualitative assessment, a rubric covering content accuracy, narrative coherence, professional tone, and formatting correctness will be developed. If IB practitioners are available for feedback, their input will supplement the rubric-based evaluation. However, the core evaluation does not depend on external participation.

## Project Plan

The project runs for thirteen weeks. Research and planning come first, followed by implementation of foundational components before integration, then evaluation and write-up. Figure 2 shows the schedule.

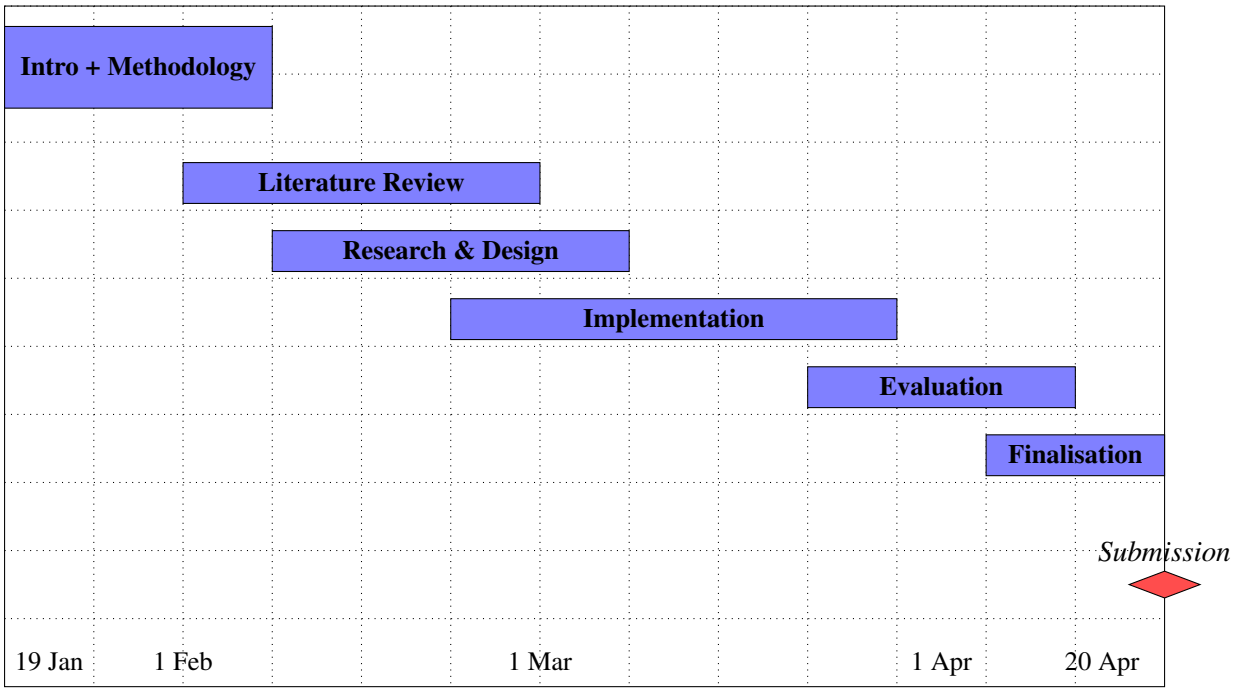


Figure 2: Project Gantt Chart (19 January to 20 April 2026)

Table 9 lists the milestones with their specific deliverables and target dates. Each milestone represents a checkpoint for assessing progress and deciding whether to continue as planned or adjust course. The dates are targets rather than commitments. If a milestone slips, the response options include compressing subsequent work, reducing scope, or accepting schedule delay.

| Phase | Date | Milestone | Deliverables |
|---|---|---|---|
| 1 | 19 Jan | Project Initiation | Repository setup, development environment configured |
| 2 | 31 Jan | Initial Chapters | Introduction, Scope, Methodology chapters complete |
| 3 | 21 Feb | Literature Review | Literature review chapter complete |
| 4 | 28 Feb | Design Complete | Architecture diagrams, API specifications, data models |
| 5 | 14 Mar | Core Components | Template analyser and data retrieval functional |
| 6 | 28 Mar | MVP Complete | End-to-end generation pipeline operational |
| 7 | 4 Apr | Testing Complete | Unit and integration tests passing, evaluation data collected |
| 8 | 11 Apr | Evaluation Complete | Results analysis and evaluation chapter written |
| 9 | 18 Apr | Document Finalised | All chapters complete, proofread, formatted |
| 10 | 20 Apr | Submission | Final report submitted via QMPlus |

Table 9: Project Milestones and Deliverables

### Risk Assessment and Mitigation

Following Boehm (1991)'s software risk management framework, Table 10 identifies key risks with likelihood and impact rated on a three-point scale.

| ID | Risk Description | Likelihood | Impact | Mitigation Strategy |
|---|---|---|---|---|
| R1 | API rate limits restrict data retrieval during development | Medium | Medium | Implement caching, use mock data for testing, stagger API calls |
| R2 | LLM outputs inconsistent or hallucinated content | High | High | Structured output schemas, validation layers, HITL review |
| R3 | Template analysis fails on complex slide layouts | Medium | High | Scope to common layouts, graceful degradation for unsupported elements |
| R4 | External API deprecation or changes | Low | High | Abstract API interactions, monitor changelogs, maintain fallback sources |
| R5 | Scope creep extends beyond POC boundaries | Medium | Medium | Strict MoSCoW prioritisation, regular scope reviews against requirements |
| R6 | Technical complexity exceeds available time | Medium | High | Iterative delivery, prioritise core pipeline, defer stretch goals |
| R7 | Data accuracy issues undermine credibility | Medium | High | Verification against manual retrieval, source attribution in outputs |
| R8 | Generated outputs violate compliance requirements | Low | High | Human review mandatory, disclaimer on all outputs, no PII processing |

Table 10: Risk Assessment and Mitigation Strategies

For high-impact risks, simple contingency rules apply. If LLM validation failures become frequent, scope will be reduced. If template extraction struggles with complex layouts, the supported template types will be narrowed. If milestones slip by more than a week, the "Could have" requirements will be dropped first.

## Ethics, Data Governance and Compliance

AI systems in financial services raise ethical concerns. Jobin et al. (2019) identified transparency, accountability, and human control as recurring principles across global AI ethics guidelines. Since generated documents can influence major business decisions, errors or bias could mislead clients or expose firms to liability.

The system keeps humans in control of analytical judgements. Generated content is marked as AI-assisted, and HITL design ensures professional review before distribution. The system prioritises accuracy over completeness, flagging uncertainty rather than presenting unverified information as fact. When the system cannot retrieve data or has low confidence in generated content, it says so explicitly rather than filling gaps with plausible-sounding fabrications.

For data governance, the system processes only data needed for PB generation, with no retention of user inputs after sessions. All external data includes source attribution. The POC does not process personally identifiable information. Company data comes exclusively from public sources. This approach sidesteps many privacy concerns while still demonstrating the core functionality.

Regarding compliance, the HITL architecture meets EU AI Act requirements for human oversight (Bank for International Settlements 2024). Professional responsibility for final content remains with the banker. The system logs generation parameters and data sources to support audit requirements. If questions arise about how a particular output was generated, the logs provide a complete trail from input to output.

### Project Tracking

Progress will be tracked using GitHub Issues with a Kanban board (Backlog, In Progress, In Review, Done). Each functional requirement maps to one or more issues, providing traceability from requirements to implementation. Changes to scope or architecture will be documented in the issue history.

This approach serves two purposes. First, it keeps the project organised during development, making it possible to see at a glance what is done, what is in progress, and what remains. Second, it creates an audit trail for the dissertation. Anyone reviewing the work can trace how implementation decisions connected to requirements and how the project evolved over time.

### Reusability

Academic projects should produce reusable artefacts where possible. Code that only works for one specific case has limited value.

The architecture separates concerns into independent modules (template analysis, data retrieval, content planning, slide assembly), each usable on its own. Someone interested only in template analysis could use that module without the rest of the system. Someone building a different kind of document generator could reuse the data retrieval layer.

All code will include Google-style docstrings, with a detailed README and architecture documentation. The codebase will be released under MIT License to minimise barriers to reuse. Configuration will be externalised so that adapting the system to different use cases does not require modifying source code.

# Preliminary Research and Design Documentation

## Literature Review

This review synthesises research across four domains: LLM productivity effects, automated presentation generation, agentic architectures, and human-in-the-loop design. Each domain is evaluated against the specific requirements of IB document generation, with gaps identified that this project addresses.

Given the project timeline and development constraints, the implementation focuses on the core generation pipeline: template analysis, content planning, and slide assembly. Agentic data retrieval from financial APIs and RAG over precedent materials are documented as stretch goals. The literature review still covers these areas because they inform the architecture and would be the natural next steps if the project continued beyond the current scope.

### LLM Productivity in Knowledge Work

Large-scale studies provide empirical evidence for LLM productivity gains. Brynjolfsson et al. (2023) studied 5,179 customer support agents and found a 14% average productivity increase, with novice workers gaining up to 34%. Noy and Zhang (2023) found professionals completed writing tasks 40% faster with 18% higher quality when using ChatGPT. Both studies show asymmetric benefits: less experienced workers gain most because AI provides access to patterns that experts have already internalised.

These findings directly apply to this project. Junior IB analysts are novice knowledge workers who spend disproportionate time on formatting rather than analysis. If AI productivity gains follow the same distribution, the target users stand to benefit most.

However, both studies measured general writing tasks. Neither addressed template-constrained document generation, where outputs must match precise visual specifications. The gap matters because IB PBs require exact font sizes, colour values, and placeholder positions. General writing assistance helps with content; it does not help with the mechanical formatting work that consumes analyst time. This project targets that specific gap.

### Automated Presentation Generation

Table 11 presents a taxonomy of presentation generation approaches, evaluated against IB requirements. The main takeaway is that every existing approach fails on at least one dimension that IB documents require. Rule-based and rigid template systems lack adaptability, LLM-based systems lack domain knowledge, and VLM-based systems lack the precision needed for exact corporate template matching.

| Approach | Mechanism | Strengths | Limitations for IB |
|---|---|---|---|
| Rule-based layout | Algorithmic text-to-slide conversion | Fast, deterministic | Poor visual quality, no domain awareness |
| Rigid templates | Fixed layouts with manual content | High visual fidelity | Cannot adapt to varying content lengths |
| LLM-based (PPTAgent) | Reference analysis + edit-based generation | Preserves source style, good content quality | Domain-generic, no financial data integration |
| Multi-agent semantic (Di Genio) | Template-constrained multi-agent pipeline | Maintains document structure | Text documents only, not PowerPoint |
| VLM-based | Visual understanding of layouts | Handles complex visual elements | Approximate values, not exact template matching |

Table 11: Taxonomy of Presentation Generation Approaches

Of these approaches, PPTAgent (Zheng et al. 2025) comes closest to what this project needs. Its two-stage approach (analyse reference, generate matching slides) aligns with IB document generation because it preserves the source deck's visual identity rather than imposing a new one. PPTAgent outperforms earlier systems on both content and design metrics using their PPTEval framework. However, PPTAgent targets general-purpose presentations. It knows nothing about financial tables, transaction comparables, or the narrative conventions that bankers expect. It also cannot connect to external data sources, so any financial figures would need to be manually inserted. Liu et al. (2024) surveyed 51 professionals and found template consistency was their top priority for LLM-generated output, which validates the template-first direction PPTAgent takes, but reinforces that domain awareness is the missing piece. Vision-language models offer an alternative path. Faysse et al. (2024) show VLMs can understand document layouts visually. However, programmatic extraction via python-pptx provides exact colour hex values and font sizes, while VLM-based approaches yield approximations. For corporate template matching where a colour being off by a shade would flag the document as non-compliant, exact values are non-negotiable. This is why this project uses programmatic extraction over visual understanding, and builds on PPTAgent's template-first philosophy while adding the IB domain layer that PPTAgent lacks.

**Agentic Architectures and Tool Use**

Wang et al. (2024) propose a framework where agents operate through planning, memory, and action loops. This maps directly to PB generation: the system plans slide structure, remembers template constraints, and acts through tool calls and slide assembly. The choice between multi-agent and single-agent architecture depends on the problem structure. Guo et al. (2024) show multi-agent

systems add value when sub-tasks require negotiation or parallel reasoning, and Li, Wang, Fang, et al. (2024) propose a five-component framework covering profiling, perception, action, memory, and communication. PB generation is sequential rather than parallel (analyse template, plan content, assemble slides), so inter-agent communication adds overhead without benefit. A single agent with tool-use capabilities is sufficient for this project's scope.

Tool use is the capability that makes this architecture work. Qu et al. (2025) identify function calling as the primary LLM-tool interface, and Li, Wang, Zhang, et al. (2025) confirm it as the most broadly applicable pattern: tool use lets the model interact with external systems without requiring those systems in its training data. In this project, the agent calls python-pptx to write slides and uses structured output schemas to maintain format consistency. Financial API calls could be added as tools in future iterations without changing the underlying architecture.

**Hallucination Mitigation and Human Oversight**

Hallucination poses the greatest risk for LLM systems in regulated industries. Huang et al. (2024) distinguish two types. Intrinsic hallucination occurs when the model contradicts information it was given in the prompt or context window. For a PB system, this could mean the model receives a company's actual revenue of $4.2 billion but outputs $4.8 billion in the slide text, introducing a factual error that looks plausible enough to survive a casual review. Extrinsic hallucination occurs when the model generates claims that have no grounding in any source. This is more dangerous for IB because fabricated metrics, invented deal precedents, or made-up market statistics could reach clients and regulators. Unlike a spelling mistake, a hallucinated financial figure can trigger compliance violations, damage client trust, or expose the firm to legal liability. The challenge is that both types of hallucination produce text that reads fluently and confidently, making them hard to catch without line-by-line verification against source data.

Alansari and Luqman (2025) group mitigation strategies into three categories: retrieval augmentation (grounding outputs in verified sources), structured output constraints (forcing the model to fill predefined schemas rather than generate freeform text), and human verification (requiring a person to review outputs before use). No single strategy eliminates hallucination. This project layers all three: financial data comes from APIs rather than LLM generation, structured JSON schemas enforce the format and content of each slide specification, and human review is mandatory before any output reaches a client. The layered approach means that even if one defence fails (for example, if the LLM ignores a schema constraint), the others should catch the error.

Human oversight has both design and regulatory dimensions. Wu et al. (2022) classify HITL approaches into data processing, interventional training, and system design. This project uses system design: the analyst defines inputs, reviews outputs, and decides whether to use them. The EU AI Act requires human oversight for AI in financial services (Bank for International Settlements 2024), making this a compliance requirement as well as a design choice. The architecture fits existing IB workflows naturally. Junior analysts already produce drafts that senior bankers revise. The system replaces the blank page, not the review process. Brynjolfsson et al. (2023)'s finding that AI helps novices most applies directly: junior analysts are the novice workers who would benefit from a formatted first draft.

### Research Gaps and Project Positioning

Three gaps emerge from this review. First, no presentation generation tool targets IB-specific conventions. Financial tables, transaction comparables, and firm templates have requirements that general-purpose tools ignore. Second, PPTAgent is domain-generic. It preserves visual design but lacks domain knowledge. Third, no system integrates agentic data retrieval with template-aware assembly. Data gathering and document creation remain separate manual steps. The literature also has methodological blind spots: productivity studies measure general tasks rather than constrained document generation, PPTAgent's evaluation uses general presentations rather than domain-specific documents with strict formatting requirements, and no study measures whether template-aware generation actually reduces analyst formatting time.

This project addresses these gaps by combining programmatic template extraction (for exact visual matching), domain-specific content planning (for IB conventions), and mandatory human review (for accuracy and compliance). The evaluation will measure template fidelity and generation speed, providing data that existing studies lack.

## System Design

This section documents the system architecture, data models, and component specifications derived from the requirements and literature review. The design prioritises template fidelity, modularity, and human oversight.

### Architecture Overview

Figure 3 shows the high-level system architecture. The design follows a pipeline pattern with three main stages: template analysis, content planning, and slide assembly.

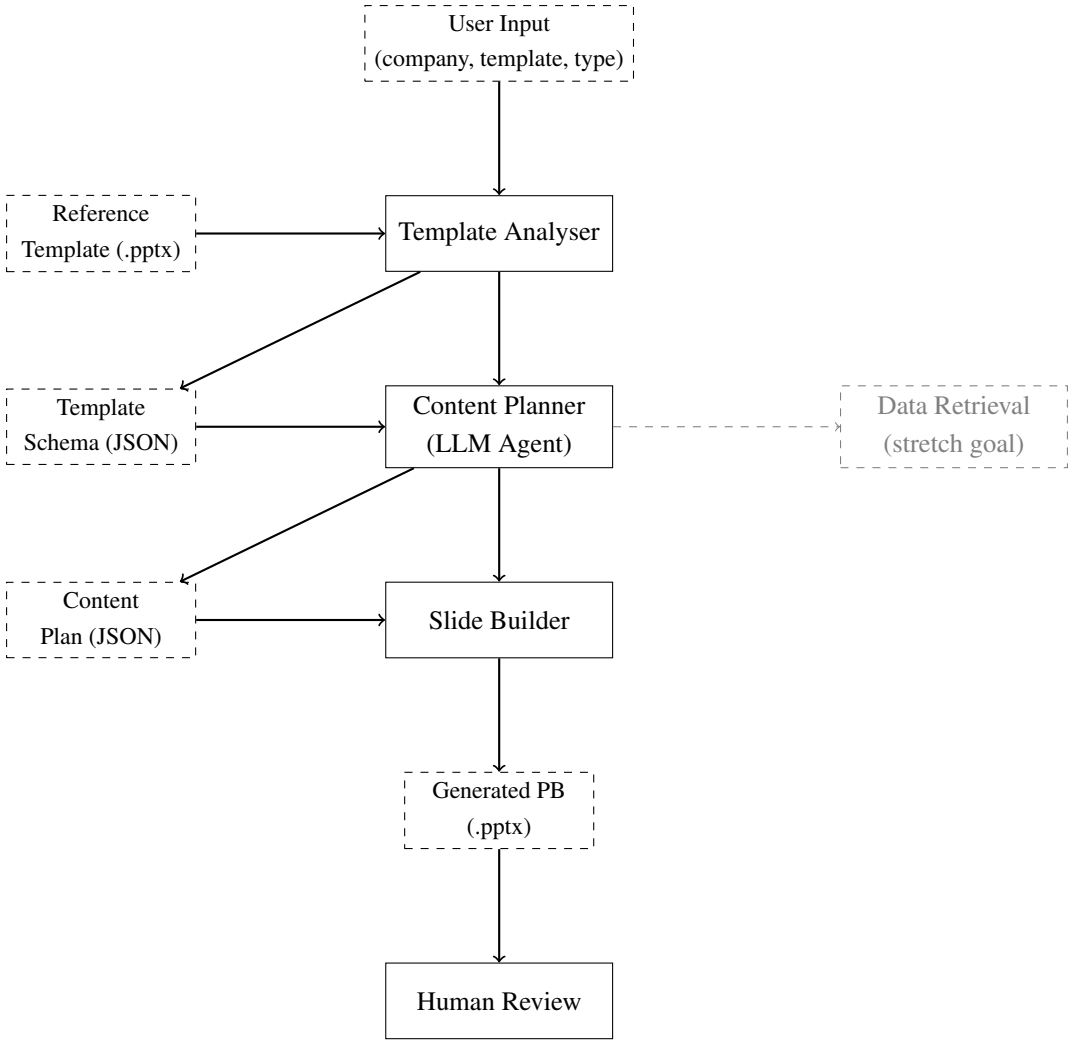


Figure 3: System Architecture showing the three-stage core pipeline. Data Retrieval is a stretch goal.

The architecture separates concerns into independent modules. The Template Analyser extracts layout patterns without generating content. The Content Planner uses an LLM to structure the narrative without touching PowerPoint files. The Slide Builder assembles files from specifications without making content decisions. This separation supports testing and allows components to be reused independently. Data Retrieval from external APIs is shown as a stretch goal that can be added later without changing the core pipeline.

### Data Models

The system uses two primary data structures that flow between components.

**Template Schema.** Captures extracted template properties including layouts, colour palettes, fonts, and placeholder specifications. Each slide layout maps placeholder indices to content types (title, body, chart, table).

**Content Plan.** Specifies the content for each slide: which layout to use, what text goes in each

placeholder, and what data to display. The LLM generates this structure; the Slide Builder consumes it.

Table 12 summarises the key fields in each data structure.

| Model | Key Fields | Purpose |
|---|---|---|
| TemplateSchema | layouts[], colours, fonts, placeholders[] | Stores extracted template properties for content planning and slide assembly |
| SlideLayout | layout_index, placeholder_map, background | Maps placeholder positions to content types for a single layout |
| ContentPlan | slides[], metadata | LLM-generated specification of slide content and structure |
| SlideSpec | layout_id, placeholders, charts[], tables[] | Content specification for a single slide |

Table 12: Primary Data Model Specifications

### Component Specifications

**Template Analyser.** Input: reference .pptx file. Output: TemplateSchema JSON. Uses python-pptx to extract slide masters, layouts, colour schemes, and placeholder positions. Identifies placeholder types (title, body, picture, chart, table) by inspecting shape properties. Extracts exact colour values as hex codes and font specifications including family, size, and weight.

**Content Planner.** Input: user parameters, TemplateSchema. Output: ContentPlan JSON. Implemented as an OpenAI Agent. Uses structured output mode to enforce JSON schema compliance. Selects appropriate layouts from the template schema based on content requirements. Plans slide sequence following IB narrative conventions (context, analysis, recommendation).

**Slide Builder.** Input: ContentPlan, TemplateSchema, reference template. Output: .pptx file. Clones the reference template to preserve master styles. Iterates through ContentPlan slides, selects layouts, and populates placeholders. Applies exact formatting from TemplateSchema. Generates charts and tables programmatically using python-pptx.

### API Design

The backend exposes a REST API via FastAPI. The primary endpoint accepts generation requests and returns job identifiers for async processing.

```
@app.post("/generate")
async def generate_pitchbook(
    request: GenerationRequest
) -> GenerationResponse:
    """
```

```
    Request fields:
    - company: str (ticker or name)
    - template_file: UploadFile (.pptx)
    - pitchbook_type: Literal["overview", "market", "transaction"]
    - sections: Optional[list[str]]

    Response fields:
    - job_id: str
    - status: Literal["queued", "processing", "complete", "failed"]
    - download_url: Optional[str]
    """
```

Listing 1: Core API endpoint specification

### Design Decisions and Trade-offs

Four design decisions reflect trade-offs between capability and implementation complexity.

**Single agent vs multi-agent.** The design uses a single agent with tools rather than multiple specialised agents. The literature review found multi-agent systems add value when sub-tasks require negotiation. PB generation is sequential, so the coordination overhead of multiple agents is not justified.

**Programmatic vs VLM template extraction.** Programmatic extraction via python-pptx provides exact values. VLM-based approaches could handle more complex visual elements but yield approximations. For template matching, exact values are essential.

**OpenAI Agents SDK vs custom orchestration.** Using the SDK creates vendor dependency but saves implementation time. For a proof of concept, this trade-off is acceptable. The component interfaces allow swapping orchestration frameworks later.

**Synchronous vs asynchronous generation.** The API uses async job processing because generation may take several minutes. Users submit requests and poll for completion rather than waiting for synchronous responses.

### Data Flow

Figure 4 illustrates how data moves through the system during a generation request. This flow matters for debugging and for tracking data provenance.

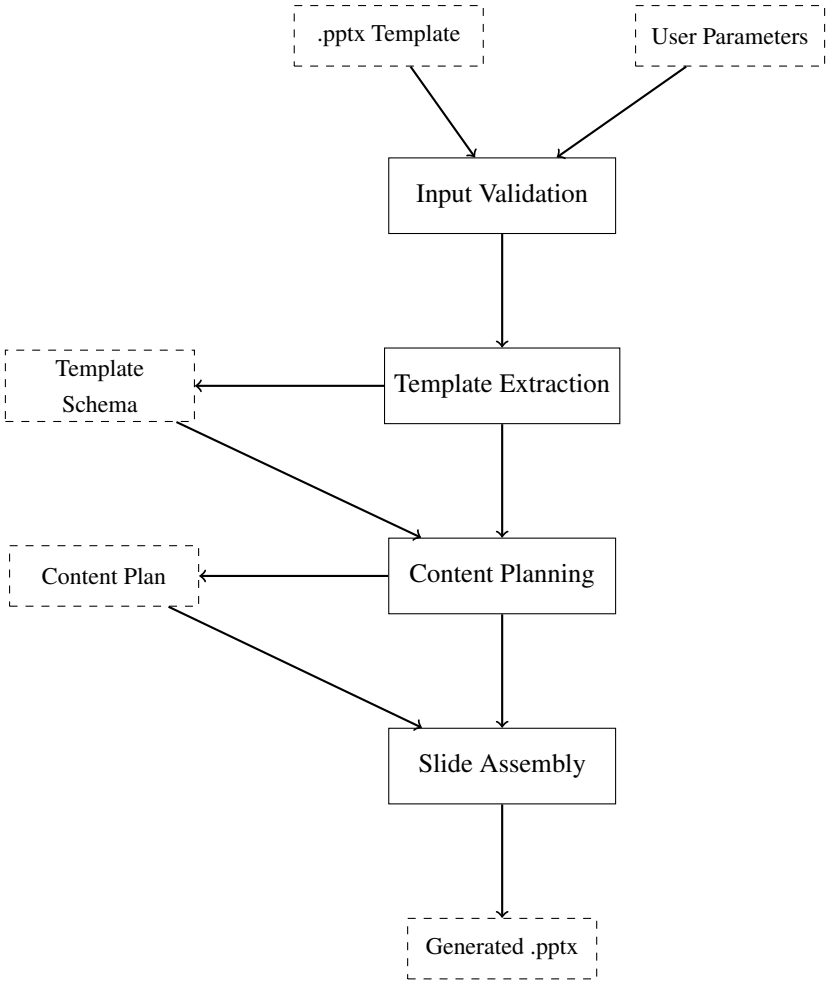


Figure 4: Data flow through the generation pipeline

The flow begins with input validation, which checks that the uploaded template is a valid .pptx file and that required parameters are present. Template extraction runs next, producing a schema that captures all layout information. The content planner receives the template schema and user parameters, producing a structured plan that the slide builder consumes to generate the output .pptx.

### User Interface Design

The frontend provides a minimal interface focused on the core workflow. Figure 5 shows the layout structure. The design keeps required inputs to a minimum and shows clear progress feedback.

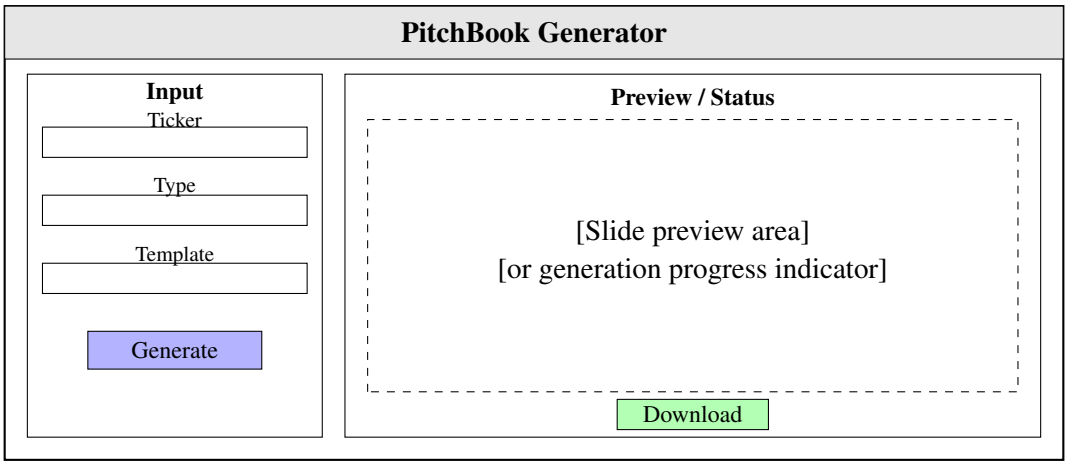


Figure 5: User interface wireframe showing input form and preview area

The interface has two main sections. The left panel contains the input form: company ticker, PB type selector, and template upload. The right panel shows either generation progress or a preview of completed slides. A download button appears when generation completes. The design deliberately omits advanced options. Users who want fine-grained control should edit the output directly rather than configuring generation parameters.

### Error Handling and Validation

The system implements validation at three levels: input validation, processing validation, and output validation.

**Input validation** checks file types, required fields, and parameter formats before processing begins. Invalid inputs return immediate error messages rather than failing partway through generation.

**Processing validation** handles failures during generation. If the LLM returns an invalid content plan, the system retries with adjusted prompts. If template extraction fails on certain slide layouts, the system logs the issue and skips unsupported layouts rather than failing entirely. Each problem is logged and surfaced to the user.

**Output validation** verifies that generated slides match template specifications. The system compares output colours, fonts, and placeholder positions against the extracted schema. Deviations beyond tolerance thresholds are logged as warnings. This validation supports the 85% template fidelity success criterion defined in the objectives.

### Security Considerations

The system handles user-uploaded files and connects to external LLM APIs, creating two attack surfaces.

For file uploads, the system validates that uploaded files are genuine .pptx archives (ZIP files with

expected internal structure) rather than accepting files based on extension alone. This prevents path traversal and zip bomb attacks. File size limits prevent denial-of-service through large uploads.

For LLM API credentials, the system uses environment variables rather than configuration files, following twelve-factor app principles. Credentials are never logged or included in error messages. Rate limiting on the generation endpoint prevents abuse through repeated requests.

### Limitations of the Design

Four limitations constrain what the current design can achieve. Being explicit about these boundaries prevents scope creep and sets appropriate expectations.

The system cannot generate charts from scratch. It can populate chart placeholders with data, but the chart type and styling must already exist in the template. Creating new chart types would require a much larger implementation effort.

Complex slide layouts with overlapping elements or custom animations may not extract correctly. The template analyser handles standard placeholder-based layouts but does not attempt to reverse-engineer arbitrary PowerPoint constructions.

The content planner has no memory across sessions. Each generation request starts fresh. The RAG capability that would enable learning from past PBs is a stretch goal, not part of the core design.

Finally, the system produces drafts, not finished documents. Human review is mandatory. The design assumes analysts will edit outputs, not use them directly. Any evaluation of the system must account for this: the goal is reducing time to first draft, not eliminating human involvement.

# Project Implementation and Outcomes

# Evaluation and Conclusions

# References


Alansari, Asmaa and Hamzah Luqman (2025). “Hallucination in Large Language Models: A Survey with Taxonomy and Mitigation Strategies”. In: *ACM Computing Surveys* 57.8. DOI: 10.1145/3703633.

APQC (2023). *Knowledge Workers’ Time Lost Due to Productivity Drains*. American Productivity & Quality Center. URL: https://www.apqc.org/about-apqc/news-press-release/apqc-survey-finds-one-quarter-knowledge-workers-time-lost-due.

Bank for International Settlements (2024). *Regulating AI in the Financial Sector: Recent Developments and Main Challenges*. 63. URL: https://www.bis.org/fsi/publ/insights63.pdf.

Basili, Victor R., Gianluigi Caldiera, and H. Dieter Rombach (1994). “The Goal Question Metric Approach”. In: *Encyclopedia of Software Engineering* 1, pp. 528–532.

BBC News (Mar. 2021). *Goldman Sachs junior bankers rebel over ‘inhumane’ 100-hour weeks*. URL: https://www.bbc.co.uk/news/business-56452494 (visited on 01/15/2025).

Boehm, Barry W. (1988). “A Spiral Model of Software Development and Enhancement”. In: *Computer* 21.5, pp. 61–72. DOI: 10.1109/2.59.

– (1991). “Software Risk Management: Principles and Practices”. In: *IEEE Software* 8.1, pp. 32–41. DOI: 10.1109/52.62930.

Brynjolfsson, Erik, Danielle Li, and Lindsey R. Raymond (2023). “Generative AI at Work”. In: *NBER Working Paper* 31161. DOI: 10.3386/w31161.

Clegg, Dai and Richard Barker (1994). “Case Method Fast-Track: A RAD Approach”. In: *Case Method Fast-Track: A RAD Approach*. Wokingham, UK: Addison-Wesley. ISBN: 978-0201624328.

Corporate Finance Institute (2025). *Investment Banking Analyst Job Description, Hours, and Salary*. URL: https://corporatefinanceinstitute.com/resources/career/investment-banking-analyst-job-description-hours-salary/ (visited on 02/24/2025).

Dalkir, Kimiz (2011). *Knowledge Management in Theory and Practice*. 2nd. Cambridge, MA: MIT Press. ISBN: 978-0262015080.

Faysse, Manuel et al. (2024). “ColPali: Efficient Document Retrieval with Vision Language Models”. In: *arXiv preprint*. arXiv: 2407.01449.

Fowler, Martin and Matthew Foemmel (2006). “Continuous Integration”. In: *ThoughtWorks*. Seminal article on CI practices. URL: https://www.martinfowler.com/articles/continuousIntegration.html.

Guo, Taicheng et al. (2024). “Large Language Model Based Multi-Agents: A Survey of Progress and Challenges”. In: *Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI)*. arXiv: 2402.01680.

Huang, Lei et al. (2024). “A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions”. In: *ACM Computing Surveys*. DOI: 10.1145/3703155.

IBM (2024). *What are Agentic Workflows?* URL: https://www.ibm.com/think/topics/agentic-workflows (visited on 11/18/2025).

Jobin, Anna, Marcello Ienca, and Effy Vayena (2019). “The Global Landscape of AI Ethics Guidelines”. In: *Nature Machine Intelligence* 1.9, pp. 389–399. DOI: 10.1038/s42256-019-0088-2.

Li, Qingyun, Chi Wang, Qi Zhang, et al. (2025). “A Review of Prominent Paradigms for LLM-Based Agents: Tool Use, RAG, and Code Interpreters”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. arXiv: 2406.05804.

Li, Shuhe, Lu Wang, Siwei Fang, et al. (2024). “A Survey on LLM-based Multi-agent Systems: Workflow, Infrastructure, and Challenges”. In: *arXiv preprint*. arXiv: 2412.13093.

Liu, Michael Xieyang et al. (2024). ““We Need Structured Output”: Towards User-centered Constraints on Large Language Model Output”. In: DOI: 10.1145/3613905.3650756.

Markus, M. Lynne (2001). “Toward a Theory of Knowledge Reuse: Types of Knowledge Reuse Situations and Factors in Reuse Success”. In: *Journal of Management Information Systems* 18.1, pp. 57–93. DOI: 10.1080/07421222.2001.11045671.

McKinsey & Company (2024). *Capturing the Full Value of Generative AI in Banking*. McKinsey & Company. URL: https://www.mckinsey.com/industries/financial-services/our-insights/capturing-the-full-value-of-generative-ai-in-banking.

McKinsey Global Institute (July 2012). *The Social Economy: Unlocking Value and Productivity Through Social Technologies*. McKinsey & Company. URL: https://www.mckinsey.com/industries/technology-media-and-telecommunications/our-insights/the-social-economy.

Noy, Shakked and Whitney Zhang (2023). “Experimental Evidence on the Productivity Effects of Generative Artificial Intelligence”. In: *Science* 381.6654, pp. 187–192. DOI: 10.1126/science.adh2586.

Qu, Changle et al. (2025). “Tool Learning with Large Language Models: A Survey”. In: *Frontiers of Computer Science* 19.8, p. 198343. DOI: 10.1007/s11704-024-40678-2. arXiv: 2405.17935.

Radhakrishna, Vedapradha et al. (2024). “Future of Knowledge Management in Investment Banking: Role of Personal Intelligent Assistants”. In: *SAGE Open* 14.4. DOI: 10.1177/20597991241287118.

Robertson, Suzanne and James Robertson (2012). *Mastering the Requirements Process: Getting Requirements Right*. 3rd. Upper Saddle River, NJ: Addison-Wesley Professional. ISBN: 978-0321815743.

Runeson, Per et al. (2012). *Case Study Research in Software Engineering: Guidelines and Examples*. Hoboken, NJ: John Wiley & Sons. ISBN: 978-1118104354.

Sommerville, Ian (2016). *Software Engineering*. 10th. Boston, MA: Pearson Education. ISBN: 978-0133943030.

Tyree, Jeff and Art Akerman (2005). “Architecture Decisions: Demystifying Architecture”. In: *IEEE Software*. Vol. 22. 2. IEEE, pp. 19–27. DOI: 10.1109/MS.2005.27.

Wall Street Oasis (2024). *Investment Banking Hours: Typical Schedule, Work-Life Balance, and Expectations*. URL: https://www.wallstreetoasis.com/resources/careers/jobs/investment-banking-hours.

Wang, Lei et al. (2024). “A Survey on Large Language Model based Autonomous Agents”. In: *Frontiers of Computer Science* 18.6, p. 186345. DOI: 10.1007/s11704-024-40231-1. arXiv: 2308.11432.

Wohlin, Claes et al. (2012). *Experimentation in Software Engineering*. Berlin: Springer. ISBN: 978-3642290435. DOI: 10.1007/978-3-642-29044-2.

Wu, Xingjiao et al. (2022). “A Survey of Human-in-the-Loop for Machine Learning”. In: *Future Generation Computer Systems* 135, pp. 364–381. DOI: 10.1016/j.future.2022.05.014.

Zheng, Hao et al. (2025). “PPTAgent: Generating and Evaluating Presentations Beyond Text-to-Slides”. In: *arXiv preprint*. arXiv: 2501.03936. URL: https://arxiv.org/abs/2501.03936.

# Appendices